

Course Code	Course Title	L	R	P	Cr
	AI-Assisted Problem Solving Using Python Programming				
Program Core					

Day 4 Session:
Advanced String
Processing &
Functions

Day 4

- String Methods
- Text Analytics
- User-defined Functions
- Function Design

Topics

- String Methods
- Text Analytics
- User-defined Functions
- Function Design

What are String Methods?

String methods are built-in functions that perform operations on strings without changing the original string.

Common String Methods

Method	Description	Example
upper()	Converts to uppercase	"python".upper()
lower()	Converts to lowercase	"HELLO".lower()
title()	Capitalizes every word	"python programming".title()
capitalize()	Capitalizes first letter	"python".capitalize()
strip()	Removes spaces	" hello ".strip()
replace()	Replaces text	"cat".replace("c","b")
split()	Splits into list	"A,B,C".split(",")
join()	Joins strings	"-".join(["A","B"])
find()	Finds substring	"Python".find("th")
count()	Counts occurrences	"banana".count("a")
startswith()	Checks prefix	"Python".startswith("Py")
endswith()	Checks suffix	"Python".endswith("on")
isdigit()	Checks numeric string	"123".isdigit()
isalpha()	Checks alphabets	"Hello".isalpha()
isalnum()	Checks letters and digits	"ABC123".isalnum()

Common String Methods : upper() & lower()

- Python's string methods allow easy transformation.
- upper(): returns a new string with all characters converted to uppercase.
- lower(): returns a new string with all characters converted to lowercase.
- These methods are often used for case-insensitive comparisons or formatting. They do not change the original string (since strings are immutable)

```
word = "PyThOn"
print(word.upper())    # Output: 'PYTHON'
print(word.lower())    # Output: 'python'
```

```
PYTHON
python
```

```
text = "Hello World!"
upper_text = text.upper()
lower_text = text.lower()
print(upper_text)      # Output: 'HELLO WORLD!'
print(lower_text)      # Output: 'hello world!'
```

```
HELLO WORLD!
hello world!
```

Common String Methods: strip()

- **strip([chars]):** Returns a new string with leading and trailing whitespace (spaces, tabs, newlines) removed. If chars is provided, it removes those characters from both ends.

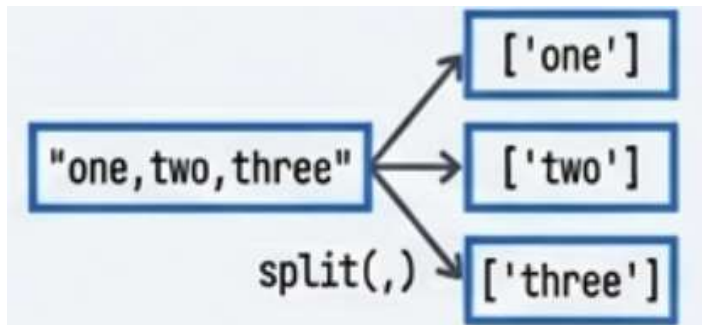


```
s = "  hello  "
print(s.strip())           # Output: 'hello'
print(">>>" + s.strip() + "<<<") # '>>>hello<<<'
path = "/home/user/"
print(path.strip("/"))     # Output: 'home/user'
```

```
hello
>>>hello<<<
home/user
```

Common String Methods: split()

- **split(sep=None)**: Splits the string into a list of substrings using sep as the delimiter (default is any whitespace).

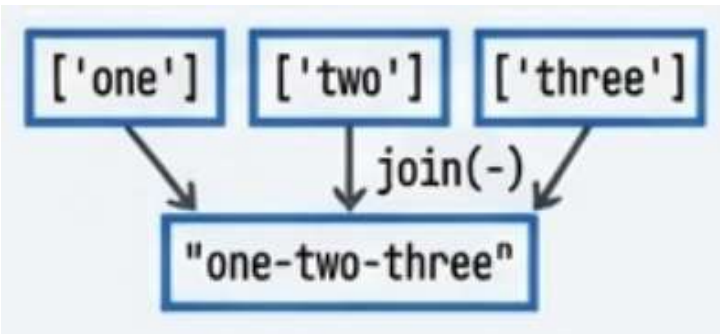


```
text = "apple,banana,cherry"
fruits = text.split(",")
print(fruits)                # Output: ['apple', 'banana', 'cherry']
sentence = "This is a test"
words = sentence.split()     # default splits on whitespace
print(words)                 # Output: ['This', 'is', 'a', 'test']
```

```
['apple', 'banana', 'cherry']
['This', 'is', 'a', 'test']
```

Common String Methods: join()

- **join(iterable):** Opposite of split(). It concatenates an iterable of strings (e.g. a list) into one string, with the original string as the separator.



```

▶ words = ["Python", "is", "awesome"]
  sentence = " ".join(words)
  print(sentence) # Output: 'Python is awesome'
  numbers = ["1", "2", "3"]
  print(",".join(numbers)) # Output: '1,2,3'
    
```

```

... Python is awesome
    1,2,3
    
```


Common String Methods: find() & replace()

- **find(sub, [start], [end]):** Searches for the substring sub and returns the lowest index where it's found, or -1 if not found. (Unlike index(), it does not raise an error if not found.)
- **replace(old, new, [count]):** Returns a new string where all (or up to count) occurrences of substring old are replaced with new.



"hello world"

↓

"hello Python"

```
text = "Hello, world"
print(text.find("world"))    # Output: 7 (index where 'world' starts)
print(text.find("x"))        # Output: -1 (not found)
print(text.find("o", 5))     # Output: 8 (find 'o' starting at index 5)

s = "Hello Hello"
print(s.replace("Hello", "Hi"))    # Output: 'Hi Hi'
print(s.replace("l", "*", 2))      # Output: 'He**o Hello' (replace first 2 'l's)
text = "spam eggs spam"
print(text.replace("spam", "ham")) # Output: 'ham eggs ham'
```

```
Hi Hi
He**o Hello
ham eggs ham
```


Multiple Methods - 1

```
sentence = "  Data Science is fun!  "
trimmed = sentence.strip()           # remove outer whitespace
print("Trimmed:", trimmed)           # 'Data Science is fun!'
words = trimmed.split()              # split into words
print("Words list:", words)          # e.g. ['Data', 'Science', 'is', 'fun!']
joined = "-".join(words)             # join with dashes
print("Joined with dashes:", joined)  # 'Data-Science-is-fun!'
index = joined.find("fun")
print("Index of 'fun':", index)       # e.g. 17
replaced = joined.replace("fun", "awesome")
print("Replaced string:", replaced)   # 'Data-Science-is-awesome!'
```

```
Trimmed: Data Science is fun!
Words list: ['Data', 'Science', 'is', 'fun!']
Joined with dashes: Data-Science-is-fun!
Index of 'fun': 16
Replaced string: Data-Science-is-awesome!
```

Multiple Methods - 2

```
greet = "Hi"  
name = "Bob"  
message = f"{greet}, {name}!"  
print(message) # 'Hi, Bob!'  
print("Length:", len(message)) # e.g. 7  
print("Uppercase:", message.upper()) # 'HI, BOB!'
```

Hi, Bob!
Length: 8
Uppercase: HI, BOB!

All string methods return a new value and do not modify the original string. Use this guide for quick reference.

Case Conversion

`capitalize()`
`casefold()`
`lower()`
`upper()`
`title()`
`swapcase()`

Searching & Checking (`is\*` methods)

<code>find()</code>	<code>isalnum()</code>
<code>rfind()</code>	<code>isalpha()</code>
<code>index()</code>	<code>isdigit()</code>
<code>rindex()</code>	<code>islower()</code>
<code>count()</code>	<code>isupper()</code>
<code>startswith()</code>	<code>isspace()</code>
<code>endswith()</code>	<code>istitle()</code>

Splitting & Joining

`split()`
`rsplit()`
`splitlines()`
`partition()`
`rpartition()`
`join()`

Trimming & Padding

`strip()`
`lstrip()`
`rstrip()`
`center()`
`ljust()`
`rjust()`
`zfill()`

Replacing & Translating

`replace()`
`translate()`
`maketrans()`

Formatting

`format()`
`format_map()`
`expandtabs()`

Method	Description
<code>capitalize()</code>	Converts the first character to upper case
<code>casefold()</code>	Converts string into lower case
<code>center()</code>	Returns a centered string
<code>count()</code>	Returns the number of times a specified value occurs in a string
<code>encode()</code>	Returns an encoded version of the string
<code>endswith()</code>	Returns true if the string ends with the specified value
<code>expandtabs()</code>	Sets the tab size of the string
<code>find()</code>	Searches the string for a specified value and returns the position of where it was found
<code>format()</code>	Formats specified values in a string
<code>format_map()</code>	Formats specified values in a string

<code>index()</code>	Searches the string for a specified value and returns the position of where it was found
<code>isalnum()</code>	Returns True if all characters in the string are alphanumeric
<code>isalpha()</code>	Returns True if all characters in the string are in the alphabet
<code>isdecimal()</code>	Returns True if all characters in the string are decimals
<code>isdigit()</code>	Returns True if all characters in the string are digits
<code>isidentifier()</code>	Returns True if the string is an identifier
<code>islower()</code>	Returns True if all characters in the string are lower case
<code>isnumeric()</code>	Returns True if all characters in the string are numeric
<code>isprintable()</code>	Returns True if all characters in the string are printable
<code>isspace()</code>	Returns True if all characters in the string are whitespaces
<code>istitle()</code>	Returns True if the string follows the rules of a title
<code>isupper()</code>	Returns True if all characters in the string are upper case
<code>join()</code>	Joins the elements of an iterable to the end of the string
<code>ljust()</code>	Returns a left justified version of the string
<code>lower()</code>	Converts a string into lower case
<code>lstrip()</code>	Returns a left trim version of the string
<code>maketrans()</code>	Returns a translation table to be used in translations

<code>partition()</code>	Returns a tuple where the string is parted into three parts
<code>replace()</code>	Returns a string where a specified value is replaced with a specified value
<code>rfind()</code>	Searches the string for a specified value and returns the last position of where it was found
<code>rindex()</code>	Searches the string for a specified value and returns the last position of where it was found
<code>rjust()</code>	Returns a right justified version of the string
<code>rpartition()</code>	Returns a tuple where the string is parted into three parts
<code>rsplit()</code>	Splits the string at the specified separator, and returns a list
<code>rstrip()</code>	Returns a right trim version of the string
<code>split()</code>	Splits the string at the specified separator, and returns a list
<code>splitlines()</code>	Splits the string at line breaks and returns a list
<code>startswith()</code>	Returns true if the string starts with the specified value
<code>strip()</code>	Returns a trimmed version of the string
<code>swapcase()</code>	Swaps cases, lower case becomes upper case and vice versa
<code>title()</code>	Converts the first character of each word to upper case
<code>translate()</code>	Returns a translated string
<code>upper()</code>	Converts a string into upper case
<code>zfill()</code>	Fills the string with a specified number of 0 values at the beginning


```
s = "hello world"

# capitalize(): Capitalizes first character, rest become lowercase
print(s.capitalize())          # Hello world

# casefold(): Aggressive lowercase conversion (useful for comparisons)
print("Straße".casefold())      # strasse

# center(): Centers string within given width using fill character
print(s.center(20, "*"))       # ****hello world*****

# count(): Counts occurrences of a substring
print(s.count("l"))            # 3

# encode(): Converts string to bytes using encoding
print(s.encode("utf-8"))       # b'hello world'

# endswith(): Checks if string ends with given suffix
print(s.endswith("world"))     # True

# expandtabs(): Replaces tabs with spaces
print("A\tB".expandtabs(4))    # A    B
```



```
# find(): Returns lowest index of substring or -1
print(s.find("world"))          # 6

# format(): Formats string using {} placeholders
print("My name is {}".format("Alice")) # My name is Alice

# format_map(): Formats string using dictionary
data = {"name": "Alice", "age": 30}
print("{name} is {age}".format_map(data)) # Alice is 30

# index(): Like find(), but raises error if not found
print(s.index("hello"))        # 0

# isalnum(): True if all characters are alphanumeric
print("Python3".isalnum())     # True

# isalpha(): True if all characters are alphabets
print("Python".isalpha())      # True

# isascii(): True if all characters are ASCII
print(s.isascii())            # True

# isdecimal(): True if all characters are decimal digits
print("123".isdecimal())       # True

# isdigit(): True if all characters are digits
print("123".isdigit())         # True
```

```
# rjust(): Right-justifies string within width
print("Hi".rjust(5, "*"))      # ***Hi

# rpartition(): Splits at last occurrence of separator
print("a-b-c".rpartition("-")) # ('a-b', '-', 'c')

# rsplit(): Splits string from right
print("a,b,c".rsplit(",", 1))  # ['a,b', 'c']

# rstrip(): Removes trailing whitespace or characters
print("hello!!!".rstrip("!"))  # hello

# split(): Splits string into list
print("Python is fun".split()) # ['Python', 'is', 'fun']

# splitlines(): Splits string at line breaks
print("A\nB\nC".splitlines())  # ['A', 'B', 'C']

# startswith(): Checks prefix
print(s.startswith("hello"))  # True

# strip(): Removes leading and trailing characters
print("***hello***".strip("*")) # hello

# swapcase(): Swaps uppercase to lowercase and vice versa
print("Hello PYTHON".swapcase()) # hELLO python

# title(): Converts string to title case
print("hello world".title())   # Hello World
```



```
# lower(): Converts string to lowercase
print("HELLO".lower())      # hello

# lstrip(): Removes leading whitespace or characters
print("  hello".lstrip())   # hello

# maketrans(): Creates translation table
table = str.maketrans("aeiou", "12345")

# translate(): Replaces characters using translation table
print("education".translate(table)) # 2d5c1t45n

# partition(): Splits at first occurrence of separator
print("a-b-c".partition("-")) # ('a', '-', 'b-c')

# removeprefix(): Removes prefix if present (Python 3.9+)
print("unhappy".removeprefix("un")) # happy

# removesuffix(): Removes suffix if present (Python 3.9+)
print("filename.txt".removesuffix(".txt")) # filename

# replace(): Replaces substring with another
print(s.replace("world", "Python")) # hello Python

# rfind(): Finds last occurrence index
print("banana".rfind("a"))    # 5

# rindex(): Like rfind(), but raises error if not found
print("banana".rindex("a"))   # 5
```

```
# upper(): Converts string to uppercase
print("hello".upper())      # HELLO

# zfill(): Pads string with zeros on left
print("123".zfill(6))       # 000123
```

Python Escape Characters

1. \ ' → Single Quote

Used to include a single quote inside a single-quoted string

```
text = 'It\'s a sunny day'
```

```
print(text)
```

Output: It's a sunny day

Explanation: \ ' allows the single quote in "It's" without ending the string

2. \\ → Backslash

Used to represent a single backslash character

```
path = "C:\\Users\\Sudhir\\Documents"
```

```
print(path)
```

Output: C:\Users\Sudhir\Documents

Explanation: Each \\ is interpreted as one literal backslash

3. \n → New Line

Inserts a line break

```
text = "Hello\nWorld"
```

```
print(text)
```

Output:

Hello

World

Explanation: \n moves the cursor to the next line

Escape Characters

Other escape characters used in Python:

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value

```
# 4. \r → Carriage Return
# Moves cursor to the beginning of the line
text = "Hello\rHi"
print(text)
# Output (typical console): Hillo
# Explanation:
# "Hello" is printed first
# \r moves cursor to start of line
# "Hi" overwrites the first two characters → Hillo

# 5. \t → Tab
# Inserts a horizontal tab space
text = "Name:\tSudhir"
print(text)
# Output: Name:    Sudhir
# Explanation: \t inserts a tab for spacing/alignment

# 6. \b → Backspace
# Removes the previous character
text = "Hello\b"
print(text)
# Output: Hello
# Explanation: \b removes the character just before it
```

```
# 7. \f → Form Feed
# Historically moves to next page; now usually acts like a new line
text = "Page1\fPage2"
print(text)
# Output (typical console):
# Page1
# Page2
# Explanation: \f is a form-feed character, often rendered as a line break

# 8. \ooo → Octal Value
# Represents a character using octal ASCII value
text = "\110\145\154\154\157"
print(text)
# Output: Hello
# Explanation:
# \110 = H, \145 = e, \154 = l, \154 = l, \157 = o (octal ASCII values)

# 9. \xhh → Hexadecimal Value
# Represents a character using hexadecimal ASCII value
text = "\x48\x65\x6C\x6C\x6F"
print(text)
# Output: Hello
# Explanation:
# \x48 = H, \x65 = e, \x6C = l, \x6C = l, \x6F = o (hex ASCII values)
```


What is Text Analytics?

Text Analytics is the process of extracting useful information from textual data.

Common Operations

- Counting words
- Counting characters
- Counting vowels
- Frequency of letters
- Finding longest word
- Searching keywords

Example 1: Count Words

Python

```
sentence = input("Enter a sentence: ")

words = sentence.split()

print("Total Words:", len(words))
```

Example 2: Count Vowels

Python

```
text = input("Enter text: ")

count = 0

for ch in text.lower():
    if ch in "aeiou":
        count += 1

print("Vowels:", count)
```

Curiosity Scenario

Suppose you are developing a Student Management System. Every time a student's marks are entered, the system must calculate the total, average, grade, and pass/fail status. If this calculation is needed in 10 different places, would you write the same code 10 times, or is there a better programming technique?

What is a Function?

A function is a reusable block of code that performs a specific task.

Advantages

Code reuse

Better readability

Easy debugging

Modular programming

Python allows two types of functions:

1. built-in functions : print(), input()..etc
2. User-defined Functions: function is defined using the **def** keyword

Example:

```
def my_function(): # user-defined function
    print("Hello from a function")
```

Some commonly used built-in functions in Python			
Input/Output	Datatype Conversion	Mathematical Functions	Other Functions
input()	bool()	abs()	__import__()
print()	chr()	divmod()	len()
	dict()	max()	range()
	float()	min()	type()
	int()	pow()	
	list()	sum()	
	ord()		
	set()		
	str()		
	tuple()		

Syntax

```
def function_name(parameters):  
    statements  
    return value
```

Creating a Function

In Python a function is defined using the `def` keyword:

Example

```
def my_function():  
    print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis

Example

```
def my_function():  
    print("Hello from a function")  
my_function() # function call
```


Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses.

You can add as many arguments as you want, just separate them with a comma.

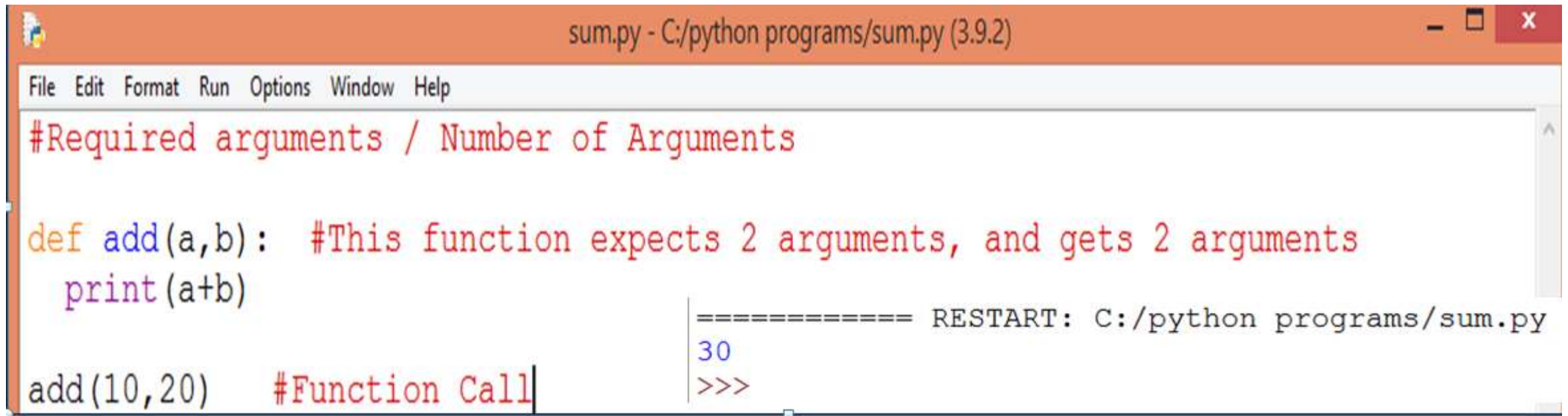
Argument Types

1. Positional Arguments / Required arguments / Number of Arguments
2. Default Arguments
3. Keyword Arguments
4. Arbitrary (variable) arguments

Required arguments / Number of Arguments:

- By default, a function must be called with the correct number of arguments.
- Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

Example: This function expects 2 arguments, and gets 2 arguments



```
sum.py - C:/python programs/sum.py (3.9.2)
File Edit Format Run Options Window Help
#Required arguments / Number of Arguments

def add(a,b): #This function expects 2 arguments, and gets 2 arguments
    print(a+b)

add(10,20) #Function Call

===== RESTART: C:/python programs/sum.py
30
>>>
```

Required arguments / Number of Arguments:

```
def name(fname, lname):  
    print(fname + " " + lname)  
  
name("Vijay", "Kumar")
```

Vijay Kumar

Error:

If you try to call the function with 1 or 3 arguments, you will get an error

Example

This function expects 2 arguments, but gets only 1:

```
def name(fname, lname):  
    print(fname + " " + lname)  
  
name("vijay")
```

```
Traceback (most recent call last):  
  File "./prog.py", line 4, in <module>  
    name("vijay")  
TypeError: name() missing 1 required positional argument: 'lname'
```

Default Parameter Value

The following example shows how to use a default parameter value. If we call the function without argument, it uses the default value.

```
def my_function(country = "India"):  
    print("I am from " + country)
```

```
my_function()  
my_function("Sweden")  
my_function("Norway")  
my_function("Brazil")  
my_function()
```

```
===== RESTART: C:/python programs/country.py
```

```
I am from India  
I am from Sweden  
I am from Norway  
I am from Brazil  
I am from India
```

Keyword Arguments

You can also send arguments with the key = value syntax. This way the order of the arguments does not matter.

```
def my_college(college1, college2, college3):  
    print("The Youngest College is: " + college3)  
  
my_college(college1 = "SREC", college2 = "SRIIT", college3 = "SRU")
```

The Youngest College is SRU

- Example 2:

```
def big_number(number3, number2, number1):  
    print("The Biggest Number is:", number1)  
  
big_number(number1 = 30, number2 = 20, number3 = 10)
```

The Biggest Number is: 30

Arbitrary Arguments, *args

- If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.
- This way the function will receive a tuple of arguments, and can access the items accordingly:

Examples

If the number of arguments is unknown, add a * before the parameter name:

```
def my_function(*students):  
    print("The Newly Joined Student is " + students[2])  
  
my_function("Ram", "Raheem", "Robert")
```

The Newly Joined Student is Robert

```
def my_function(*students):  
    print("The Newly Joined Student is Marks are ",students[1])  
  
my_function(80,90,85)
```

The Newly Joined Student is Marks are 90

Arbitrary Keyword Arguments, **kwargs

- If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ** before the parameter name in the function definition.
- This way the function will receive a *dictionary* of arguments, and can access the items accordingly

Example

If the number of keyword arguments is unknown, add a double ** before the parameter name

```
def my_function(**name):  
    print("My last name is " + name["lname"])  
  
my_function(fname = "vijay", lname = "kumar")
```

```
My last name is kumar
```

Formal vs Actual arguments in python

- Formal arguments are identifiers used in the function definition to represent corresponding actual arguments.
- Actual arguments are values(or variables)/expressions that are used inside the parentheses of a function call.

Formal vs Actual Arguments

Function Definition

```
def add(a, b):  
    return a + b
```

Formal Arguments
(or Parameters)

Function Call

```
add(2, 3)
```

Actual Arguments
(or Parameters)

Return Values

- A **return statement** is used to end the execution of the function call and “returns” the result back to the caller
- To let a function return a value, use the return statement

```
def my_function(x):  
    return 5 * x  
  
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```

```
15  
25  
45
```

Function Design

Good Function Design Principles

- ✓ Single Responsibility
- ✓ Meaningful Function Name
- ✓ Reusable Code
- ✓ Minimal Parameters
- ✓ Return Values Instead of Printing

Modular Program

</> Python

```
def get_marks():  
    return int(input("Enter Marks: "))  
  
def calculate_grade(mark):  
    if mark >= 90:  
        return "A"  
    elif mark >= 75:  
        return "B"  
    elif mark >= 60:  
        return "C"  
    else:  
        return "Fail"  
  
def display(result):  
    print("Grade:", result)  
  
marks = get_marks()  
grade = calculate_grade(marks)  
display(grade)
```



Thank you